

StuGo CO2 Explorer

Rapport de projet complet

Contexte, méthodologie, normes, architecture, classes et fonctionnalités

Remi Kalkan | [Formation / poste] | August 2026

Ce document présente, de bout en bout, le projet StuGo CO2 Explorer : le besoin à l'origine du logiciel, la démarche suivie pour le réaliser, les normes imposées par le service informatique, les outils utilisés, l'architecture logicielle retenue, l'inventaire des classes et leur rôle, ainsi que le fonctionnement du logiciel du démarrage à la fermeture.

Les noms de personnes, de service et les dates précises n'ont pas été renseignés dans cette version et doivent être complétés avant remise ([le client], [le service informatique], [dates exactes]).

Sommaire

- 1. Contexte et objectif du projet
- 2. Déroulement du projet et méthodologie
- 3. Normes et exigences du service informatique
- 4. Outils et technologies utilisés
- 5. Architecture logicielle
- 6. Patterns de conception utilisés
- 7. Inventaire détaillé des classes
- 8. Fonctionnalités du logiciel, de A à Z
- 9. Diagramme de classes (Mermaid)
- 10. Bilan et conclusion

1. Contexte et objectif du projet

[Le client] a exprimé le besoin d'un outil adapté à son travail quotidien : suivre et comparer les émissions de CO2 générées par la mobilité des étudiants (déplacements internationaux, échanges, stages) au sein de son établissement. Les données existaient déjà sous forme de fichiers Excel remplis manuellement, faculté par faculté, mais aucun outil ne permettait de les centraliser, de les filtrer ni de les visualiser simplement.

L'objectif du projet a donc été de concevoir une application de bureau, simple à prendre en main, capable de :

- importer directement les fichiers Excel/CSV existants sans changer le format de saisie du terrain ;
- agréger et nettoyer automatiquement les données (doublons, feuilles vides, valeurs manquantes) ;
- filtrer les données par faculté, zone d'émission, pays, nombre d'étudiants ou volume de CO2 ;
- visualiser les résultats sous forme de graphiques 2D et 3D exploitables en réunion ou en rapport ;
- comparer plusieurs fichiers/facultés entre eux ;
- fonctionner hors ligne, sans dépendance à un serveur ou à une connexion internet, pour des raisons de confidentialité des données étudiantes et de contraintes du poste de travail visé.

Le logiciel — StuGo CO2 Explorer — répond à ce besoin sous la forme d'une application Windows packagée (installateur), accompagnée d'un second outil autonome (Admin_Logs) permettant de consulter les journaux techniques générés par l'application principale, utile en cas d'anomalie signalée par un utilisateur.

2. Déroulement du projet et méthodologie

Le projet s'est déroulé de janvier à mai 2026, selon une démarche itérative proche d'une méthode Agile : le développement a avancé par cycles courts (sprints), chacun clôturé par un rapport dédié (Rapport_Sprint_v1_v2_v3, v4_v5, v6, puis Rapport_Projet_Sprint_v7), permettant de valider régulièrement les fonctionnalités livrées avec [le client] plutôt que d'attendre la fin du projet pour un unique retour.

Période	Étape	Détail
Janvier 2026	Recueil du besoin	Réunion(s) avec [le client] pour comprendre l'usage réel des fichiers Excel existants, les zones d'émission CO2, et les attentes en matière de filtres et de graphiques.
Janvier – avril 2026	Développement itératif (sprints v1 à v7)	Construction progressive : import de fichiers, tableau de données, graphiques 2D, filtres dynamiques, graphiques 3D, page de comparaison multi-fichiers, thèmes et paramètres, session persistante.
Tout au long du projet	Réunions de suivi avec [le client]	Points réguliers pour ajuster l'application à son besoin réel (colonnes affichées, types de graphiques utiles, filtres manquants), consignés dans les rapports de sprint et de tests.
Avril 2026	Fin du développement fonctionnel	Le programme est fonctionnellement terminé : toutes les fonctionnalités demandées sont implémentées et testées (cf.

		rapports de tests v1_v3, v4_v5, v6).
Avril – mai 2026	Sollicitation du service informatique	Le service informatique, responsable de la validation avant tout déploiement sur les postes de l'établissement, exige une démonstration du logiciel avant autorisation.
Mai 2026	Organisation de la démonstration	Plusieurs échanges de mails sont nécessaires pour fixer un créneau commun ; le calendrier glisse à plusieurs reprises en raison d'indisponibilités réciproques (congés, autres priorités), alors que le logiciel est fini depuis avril.

Ce décalage illustre un point important pour la suite du projet : le retard constaté entre avril et mai n'est pas un retard de développement, mais un délai administratif/organisationnel lié à la coordination des plannings entre les parties prenantes. Le logiciel livré à la démonstration est identique à celui achevé fin avril, aux corrections mineures près relevées lors des tests.

3. Normes et exigences du service informatique

Avant tout déploiement sur les postes de l'établissement, [le service informatique] impose un socle d'exigences techniques, indépendamment du contenu fonctionnel du logiciel. Le tableau suivant récapitule ces exigences et la manière dont le projet y répond concrètement.

Exigence	Réponse apportée dans le projet
Fonctionnement hors ligne	Aucun appel réseau applicatif dans le code source : import, calculs, rendu graphique et persistance sont intégralement locaux.
Pas d'écriture dans Program Files	L'installateur (Inno Setup) installe le programme sous {localappdata}\Programs\StuGo CO2 Explorer ; aucune écriture n'est faite sous Program Files.
Données utilisateur isolées du programme	Préférences, session et logs de l'exécutable sont écrits dans le profil utilisateur, sous %LOCALAPPDATA%\StuGoCO2 — jamais dans le dossier d'installation.
Pas de droits administrateur requis	L'installateur est configuré avec un niveau de privilège minimal (utilisateur standard) ; aucune élévation n'est nécessaire à l'installation ni à l'exécution.
Registre limité à l'utilisateur courant	Les seules clés de registre créées le sont sous HKEY_CURRENT_USER (chemin d'installation, numéro de version) ; aucune clé HKEY_LOCAL_MACHINE.
Traçabilité / journalisation	Chaque action significative (import, filtre, export, changement de session) est journalisée dans un fichier de log quotidien, consultable via l'outil Admin_Logs fourni séparément.
Code source complet et documenté	Le dépôt remis contient l'intégralité des sources Python, le diagramme de classes (architecture.mmd), les scripts de build/installateur et la documentation utilisateur/technique.

Dépendances identifiées et versionnées	Toutes les bibliothèques tierces utilisées sont listées avec leur version minimale (voir section 4) ; aucune dépendance non déclarée.
Démonstration avant déploiement	Le service informatique exige une présentation du logiciel en conditions réelles avant toute autorisation de déploiement sur le parc informatique.

4. Outils et technologies utilisés

4.1. Langage et bibliothèques

Outil	Rôle dans le projet	Version minimale
Python	Langage de développement de l'application et des scripts de build/rapports.	3.10+
PyQt6	Framework d'interface graphique (fenêtres, widgets, signaux/slots).	6.4+
pandas	Chargement, nettoyage, agrégation et filtrage des données tabulaires.	2.0+
NumPy	Calculs numériques utilisés par les moteurs de rendu 2D/3D.	—
matplotlib	Génération de tous les graphiques 2D et 3D (bar, pie, donut, aires, treemap, cubes).	3.7+
openpyxl	Lecture des fichiers Excel (.xlsx/.xls) via pandas.	3.1+
squarify	Génération des graphiques treemap.	0.4+
python-docx	Génération automatisée des rapports de sprint, de tests et technique du projet.	—

4.2. Outils de fabrication et de livraison

Outil	Rôle
PyInstaller	Empaquette l'application Python et ses dépendances en exécutable Windows autonome.
Inno Setup	Génère l'installateur Windows (StuGoCO2_Setup) à partir de l'exécutable PyInstaller.
Git	Gestion de versions du code source tout au long des sprints.

4.3. Méthodologie de conduite de projet

Le développement a suivi une logique itérative inspirée d'Agile/Scrum : découpage en sprints successifs (v1 à v7), chacun documenté (rapport de sprint + rapport de tests dédié), avec des points de validation réguliers auprès de [le client] plutôt qu'une seule livraison finale. Cette approche a permis d'ajuster les fonctionnalités (filtres, types de graphiques, ergonomie) au fil de l'eau, en fonction des retours d'usage réel.

5. Architecture logicielle

Le projet suit une architecture en couches inspirée de la Clean Architecture, avec une règle de dépendance stricte : chaque couche ne connaît que celle immédiatement en dessous, jamais l'inverse. Cela permet, par exemple, de changer la façon dont les données sont stockées (JSON aujourd'hui) sans toucher à l'interface graphique.

Couche	Répertoires	Responsabilité
Présentation	presentation/pages, sidebar, widgets, theme	Construire l'interface PyQt6 (pages, barres latérales, composants) et relier les interactions utilisateur aux services.
Application	app/commands, app/events, app/services	Orchestrer les cas d'usage : commandes réversibles, bus d'événements, façades de service (données, graphiques, export).
Domaine	domain/repositories, domain/value_objects	Définir les contrats (interfaces) et les objets métier, indépendants de Qt et du mode de stockage.
Infrastructure	infrastructure/extractors, models, persistence	Lire les fichiers Excel/CSV, adapter pandas à Qt, persister les données en JSON.
Rendu	rendering/factory, strategies, axes, geometry	Sélectionner et exécuter les rendus graphiques 2D/3D selon le type de graphique demandé.
Transversal	shared/constants, scaling, logging, paths	Centraliser les couleurs, tailles (adaptées au DPI de l'écran), chemins et journalisation, utilisés par toutes les couches.

Flux de démarrage : main.py crée la QApplication, appelle app/bootstrap.py (qui initialise l'échelle DPI, le thème et la feuille de style globale), affiche un écran de démarrage (splash), puis ouvre la fenêtre principale (MainWindow) qui assemble les pages, les barres latérales et les services applicatifs.

6. Patterns de conception utilisés

Pattern	Où il est utilisé	Pourquoi
Singleton	ThemeManager, ActionLogger, FileLogger, DataService, EventBus, SessionRepository, PreferencesRepository, AppState, UIMediator (méthode instance()).	Garantir une seule instance partagée d'un service central, accessible partout sans le faire transiter en paramètre.
Repository	SessionRepository / PreferencesRepository derrière ISessionRepository / IPreferencesRepository.	Isoler le code métier du détail de stockage (ici JSON dans AppData).
Factory	ExtractorFactory (choix Excel/CSV), RendererFactory (choix 2D/3D).	Le code appelant ne dépend pas du choix concret de l'implémentation, seulement de son résultat.
Strategy	Renderer2D / Renderer3D sélectionnés selon le mode et le type de graphique.	Changer d'algorithme de rendu sans modifier le code qui l'appelle.

Command	Command, LoadFilesCommand, ApplyFilterCommand, CommandHistory.	Encapsuler une action utilisateur (charger un fichier, filtrer) pour pouvoir l'annuler.
Observer / Publish-Subscribe	EventBus (publish/subscribe) et signaux Qt (theme_changed, filter_changed...).	Découpler les composants : une page réagit à un événement sans connaître qui l'a déclenché.
Adaptateur	PandasModel adapte un DataFrame pandas à QAbstractTableModel.	Réutiliser pandas pour les calculs tout en affichant les données dans un tableau Qt natif.
Façade	DataService, ChartService, ExportService, MainWindow.	Offrir une API simple à l'interface, qui masque la complexité de plusieurs sous-systèmes.
Médiateur	MainWindow et ui_mediator.py.	Centraliser la coordination entre pages et barres latérales plutôt que de les faire communiquer directement entre elles.
Value Object	ChartConfig (dataclass gelée/immuable), ColorValue.	Représenter une configuration ou une couleur comme une valeur immuable, sans effets de bord.
Builder	StylesheetBuilder.	Construire la feuille de style QSS complète à partir des couleurs et tailles courantes.

7. Inventaire détaillé des classes

Cette section détaille, couche par couche, les classes qui portent une responsabilité architecturale identifiable, leur utilité et leurs méthodes principales. Le détail exhaustif des attributs est disponible dans le diagramme Mermaid (section 9).

7.1. Domaine (domain/)

Classe	Utilité	Méthodes principales
ChartConfig	Dataclass immuable décrivant une configuration de graphique complète (type, colonnes, mode 3D, angle de vue).	default(), with_type(), with_3d(), as_2d()
ColorValue	Objet-valeur représentant une couleur et ses transformations (assombrir, éclaircir, teinte, contraste).	from_rgb(), darkened(), lightened(), hue_shifted(), to_rgb(), contrast_text()
ISessionRepository	Interface (Protocol) définissant le contrat qu'un dépôt de session doit respecter, indépendamment du stockage réel.	load_session(), save_session(), clear_session(), get_session_dir()
IPreferencesRepository	Interface équivalente pour les préférences utilisateur.	load(), save()

7.2. Infrastructure (infrastructure/)

Classe	Utilité	Méthodes principales
--------	---------	----------------------

SessionRepository	Implémente ISessionRepository : persiste la session (fichiers chargés, filtres) dans un fichier JSON du profil utilisateur.	instance(), load_session(), save_session(), clear_session(), get_session_dir()
PreferencesRepository	Persiste les préférences (thème, colonnes visibles) en JSON.	instance(), load(), save()
ExtractorFactory	Choisit dynamiquement l'extracteur (Excel ou CSV) selon l'extension du fichier importé.	get(extension), register(ext, handler), supported()
ExcelExtractor / CsvExtractor	Lisent, valident et normalisent les fichiers d'entrée : vérification des colonnes attendues, calcul des totaux CO2 par zone, exclusion des feuilles 'template' et des doublons.	extract_all_sheets() / extract_from_csv()
PandasModel	Adapte un DataFrame pandas à l'interface QAbstractTableModel, pour l'afficher dans un tableau Qt avec tri natif.	rowCount(), columnCount(), data(), headerData(), sort(), update_df()

7.3. Application (app/)

Classe	Utilité	Méthodes principales
EventBus	Bus d'événements central basé sur les signaux PyQt6 ; découple les composants entre eux.	instance(), publish(), subscribe(), unsubscribe()
AppState	Gère l'état global de l'application (chargement, prêt, erreur).	instance(), current, transition(), is_ready(), is_loading()
Command / CommandHistory	Encapsulent une action utilisateur réversible et son historique d'annulation.	execute(), undo(), can_undo(), undo_last(), clear()
LoadFilesCommand / ApplyFilterCommand	Commandes concrètes : charger des fichiers (annulable) et appliquer les filtres actifs.	execute(), undo(), can_undo()
DataService	Façade singleton de toutes les opérations sur les données : extraction, mise à jour du modèle, filtrage, dédoublonnage.	instance(), load_files(), remove_file(), filtered_df(), reset()
ChartService	Façade de rendu graphique : choisit et déclenche le bon renderer (2D/3D).	render()
ExportService	Exporte le graphique affiché en image (PNG/PDF/SVG) ou les données en CSV.	export_png(), export_csv()

7.4. Rendu graphique (rendering/)

Classe / module	Utilité	Méthodes principales
RendererFactory	Sélectionne la fonction de rendu (2D ou 3D) selon le mode demandé par l'utilisateur.	get(mode_3d), render()
Renderer2D	Dessine les graphiques 2D : barres,	render_chart(), render_bar(),

	camembert, donut, aires, treemap.	render_pie(), render_area(), render_treemap()
Renderer3D	Dessine les graphiques 3D : barres-cubes, camembert en volume, aires empilées ; gère aussi la mise en page (marges, zoom, aspect des axes) propre à la 3D.	render_chart_3d(), render_bar_3d(), render_pie_3d(), render_area_3d(), setup_3d_axes(), zoom_3d_axes(), finalize_3d_layout()
Cube3D	Construit et dessine les cubes 3D utilisés par les graphiques en barres ; chaque cube est une collection indépendante triée par profondeur, pour éviter que des cubes se chevauchent visuellement.	build_cube_faces(), cube_center(), draw_cubes_batch()
Pie3D	Construit et dessine les parts de camembert/donut en volume, avec le même principe de tri par profondeur que Cube3D.	build_pie3d_faces(), draw_pie3d_all()

7.5. Présentation (presentation/)

Classe	Utilité	Méthodes principales
MainWindow	Fenêtre principale : assemble la barre de navigation, les barres latérales, les pages, et fait le lien (médiateur) entre les signaux de l'interface et les services applicatifs.	build_ui(), nav(), load_files(), apply_filters(), save_session(), closeEvent()
SplashScreen	Écran de démarrage affiché pendant l'initialisation de l'application.	__init__()
HomePage	Tableau de bord d'accueil : statistiques globales, horloge, raccourcis vers les autres pages.	update_stats()
ImportPage	Page d'import des fichiers Excel/CSV, avec liste des fichiers chargés et retrait individuel.	add_file()
TablePage	Affiche les données sous forme de tableau triable, avec export CSV.	update_df()
ChartPage	Page de graphique unique avec ses contrôles (type, axes, mode 2D/3D, angle de vue) et export PNG.	update_data(), refresh(), save()
ComparisonPage	Compare plusieurs fichiers/facultés : vues côte-à-côte, superposée, en grille, ou empilée en 3D par fichier.	update_data(), render_unified(), render_grid(), render_stacked_3d()
SettingsPage	Réglages : choix de thème, éditeur de palette personnalisée, curseur de souris, gestion de session.	set_app(), restore_cursor()
HelpPage	Page d'aide décrivant l'utilisation du logiciel.	—
FilterSidebar	Barre de filtres dynamiques : faculté, zone CO2, pays, plage d'étudiants, plage de CO2,	apply(), reset_filters(), get_state(), restore_state()

	masquage des lignes à zéro étudiant.	
Sidebar / NavButton	Barre de navigation latérale entre les pages.	set_page(), set_active()
LogSessionPanel	Historique des actions utilisateur et gestion de la session (sauvegarder/reprendre).	refresh(), update_session_info()
ThemeManager	Applique les thèmes de couleur à toute l'application et notifie les widgets d'un changement.	instance(), apply(), register_preset()
ChartControls / StepWidget	Barre d'outils du graphique (sélecteurs, curseurs d'angle 3D) et compteur numérique réutilisable avec boutons +/-.	set_compat_msg() / value(), setValue()

7.6. Utilitaires transversaux (shared/)

Classe / module	Utilité
ActionLogger	Historique en mémoire des actions utilisateur (import, filtre, export), consulté par le panneau de session.
FileLogger	Écrit un fichier de log quotidien (log-AAAA-MM-JJ.txt), thread-safe.
constants.py (C, ZONES, COLUMN_LABELS...)	Source unique de vérité pour les couleurs, les zones d'émission CO2 et les libellés de colonnes.
scaling.py (S)	Calcule toutes les tailles de l'interface en fonction du DPI de l'écran, pour un rendu net sur tout type d'affichage.

8. Fonctionnalités du logiciel, de A à Z

8.1. Démarrage

Au lancement, l'application calcule l'échelle DPI de l'écran, applique le thème visuel enregistré et affiche un écran de démarrage (splash) pendant l'initialisation, avant d'ouvrir la fenêtre principale. La dernière session (fichiers chargés, filtres actifs) est restaurée automatiquement si elle existe.

8.2. Page d'accueil

Tableau de bord présentant en un coup d'œil le nombre de fichiers chargés, le nombre d'enregistrements, le nombre d'exports effectués, une horloge en temps réel, et des raccourcis vers les autres pages.

8.3. Import de données

L'utilisateur sélectionne un ou plusieurs fichiers Excel (.xlsx/.xls) ou CSV. Chaque feuille Excel est validée indépendamment : structure de colonnes attendue par zone d'émission (5 zones, de moins de 0,5 tCO2e à plus de 3 tCO2e), présence des champs pays et tCO2e. Les feuilles nommées 'template' sont ignorées automatiquement, et un fichier déjà chargé (même faculté, même feuille) n'est pas dupliqué. Toute anomalie est signalée dans les logs.

8.4. Tableau de données

Affiche l'ensemble des enregistrements chargés et filtrés dans un tableau triable par colonne, avec export possible au format CSV.

8.5. Graphiques (page Graphique)

Permet de construire un graphique unique à partir des données filtrées, avec choix du type (barres, camembert, donut, aires, treemap), des colonnes en abscisse/ordonnée, du regroupement, et du nombre d'éléments affichés (top N). Un bouton bascule entre le rendu 2D et un rendu 3D en volume (barres-cubes, camembert en volume, aires empilées), avec des curseurs pour ajuster l'angle de vue (élévation et azimuth) et un zoom automatique qui garde le graphique lisible sans qu'aucune forme n'en cache une autre. Le graphique final peut être exporté en image (PNG/PDF/SVG).

8.6. Comparaison multi-fichiers

Permet de comparer plusieurs fichiers/facultés entre eux, sous plusieurs formes : superposition unifiée sur un même graphique, vue en grille (un graphique par fichier), vue côte-à-côte, ou vue empilée en 3D (une colonne par fichier, empilant les valeurs par catégorie). Une recherche et des cases à cocher permettent de sélectionner rapidement les fichiers à comparer.

8.7. Filtres dynamiques

Une barre latérale de filtres, commune aux pages Graphique et Comparaison, permet de restreindre les données affichées par faculté, par zone d'émission CO2, par recherche de pays, par plage de nombre d'étudiants (min/max), par plage de CO2 total (min/max), et de masquer les lignes à zéro étudiant. Les filtres peuvent être réinitialisés en un clic, et sont conservés d'une session à l'autre.

8.8. Paramètres et personnalisation

La page Paramètres permet de choisir un thème de couleur parmi plusieurs pré-réglages, de créer une palette personnalisée (éditeur de couleurs dédié), de personnaliser le curseur de la souris, et d'effacer la session en cours si besoin.

8.9. Session, préférences et journalisation

À chaque fermeture, la session (fichiers chargés, filtres actifs) et les préférences (thème, curseur) sont sauvegardées automatiquement dans des fichiers JSON situés dans le profil utilisateur (%LOCALAPPDATA%\StuGoCO2), et restaurées au lancement suivant. Chaque action importante est consignée dans un fichier de log quotidien.

8.10. Outil Admin — Analyseur de logs

Un second exécutable, totalement indépendant de l'application principale, permet de charger un dossier de logs, de les filtrer par mot-clé, par module ou par date, de les colorer par niveau (erreur, avertissement, information...), et d'isoler les lignes qui ne respectent pas le format attendu — utile pour diagnostiquer un problème signalé par un utilisateur sans avoir à ouvrir le code source.

9. Diagramme de classes (Mermaid)

Le diagramme complet est fourni séparément dans le fichier architecture.mmd (format Mermaid), à ouvrir dans Mermaid Live Editor, GitHub, ou tout éditeur compatible (VS Code avec l'extension Mermaid, par exemple). Il

couvre l'ensemble des couches du projet : domaine, infrastructure, application, rendu, présentation et l'outil admin. Un extrait représentant les relations structurantes est reproduit ci-dessous.

classDiagram

```
class ChartConfig { +with_type() +with_3d() +as_2d() }
class ColorValue { +from_rgb() +darkened() +lightened() +to_rgb() }
class ISessionRepository { <<interface>> }
class SessionRepository { +instance() +load_session() +save_session() +clear_session() }
class PreferencesRepository { +instance() +load() +save() }
class ExtractorFactory { +get(extension) +register() +supported() }
class DataService { +instance() +load_files() +remove_file() +filtered_df() +reset() }
class EventBus { +instance() +publish() +subscribe() +unsubscribe() }
class CommandHistory { +execute() +undo_last() +clear() }
class RendererFactory { +get(mode_3d) +render() }
class Renderer3D { +render_chart_3d() +setup_3d_axes() +finalize_3d_layout() }
class Cube3D { +build_cube_faces() +cube_center() +draw_cubes_batch() }
class MainWindow { +build_ui() +apply_filters() +save_session() +closeEvent() }
SessionRepository ..|> ISessionRepository
PreferencesRepository ..|> IPreferencesRepository
DataService --> ExtractorFactory
DataService --> PandasModel
CommandHistory --> Command
LoadFilesCommand --|> Command
ApplyFilterCommand --|> Command
DataService --> EventBus
ChartService --> RendererFactory
RendererFactory --> Renderer2D
RendererFactory --> Renderer3D
Renderer3D --> Cube3D
Renderer3D --> Pie3D
MainWindow --> DataService
MainWindow --> SessionRepository
MainWindow --> EventBus
```

10. Bilan et conclusion

Le développement s'est déroulé de janvier à avril 2026 selon une démarche itérative documentée (sprints v1 à v7), avec des points de validation réguliers auprès de [le client], aboutissant à un logiciel fonctionnellement complet et testé dès la fin avril. Le logiciel répond aux exigences techniques du service informatique : fonctionnement hors ligne, données utilisateur isolées dans le profil (AppData), absence de droits administrateur requis, registre limité à l'utilisateur courant, et code source livré dans son intégralité.

Le délai supplémentaire entre la fin du développement (avril) et la démonstration effective au service informatique (mai) est d'origine organisationnelle : plusieurs échanges de mails ont été nécessaires pour trouver un créneau commun, retardé par des indisponibilités de part et d'autre. Ce délai n'a pas remis en cause l'état d'achèvement du logiciel, qui était prêt à être présenté dès la fin du développement.

En synthèse : le besoin exprimé par [le client] a été traduit en une application complète, structurée selon une architecture en couches propre et documentée, mettant en œuvre des patterns de conception reconnus (Singleton, Repository, Factory, Strategy, Command, Observer, Façade, Médiateur, Value Object). Le projet reste à valider formellement par le service informatique lors de la démonstration, étape administrative distincte de l'achèvement technique du logiciel.